Speaking script — WireGuard for Apache NuttX (Community Over Code, Glasgow)

Deck: `coc-glasgow-slides.html` (and the version on the official template, `coc-glasgow-reveal/`), 43 slides, English. This file is **made by** `export_en_script.py` from the speaker notes in the deck (press `N` there to see them). The "Say" text below *is* those notes. Do not edit this file by hand. Change the notes in the deck and run the script again, or the two will stop matching. The Japanese practice script uses the same slide order.

Total: about 33:32, including a 90-second demo. The times add up from the start ("you should be at about this point"), at 150 words per minute. This is longer than a 30-minute slot, so practice to finish the main talk in 27–28 minutes, and keep the cut list below ready.

The most important part is slides 31–33 (testing). Together they take 5.6 minutes of the talk, and they hold the claims that a reviewer will really check. Their narration was already trimmed (2026-10-11): process detail such as the memory map, the usrsock steps and the hostfs detail moved to the Q&A file. Do not cut their claims or their caveats (FLAT / QEMU / sim, pre-rebase, PROTECTED 2 of 8 open, one scripted 4-CPU run, the 75 s watch) any further. Cut before them or after them. They are one story: the first is where it runs, the second is what "it runs" does not prove, and the third is the one place where I had to decide instead of test. If you split them, the story breaks.

Do not cut slides 4–15 either (About Me to Architecture). What NuttX is, where it is used, Sony's history with it, how I met it, and why that led to WireGuard: everything after that depends on this.

If time is short, cut in this order:

1. Slide 40 (Giving Back). It helps the talk, but the talk does not depend on it. If you cut it, answer it in Q&A if someone asks.
2. Slide 39 (Other CPUs, the portability table). It adds support, but it is not the main story.
3. Slide 38 (Running It for Real).

If the live demo stops, switch to the recording right away. **The agenda slides (3, 11, 16, 19, 27, 34, 37) are a ten-second pause each**: point at the highlighted line and pause. Do not read the list. If you have no time at all, skip them without a word. They save almost no time, so cut the three slides above instead.

Check before the talk: the GitHub star numbers on slide 8 change. Check them again on the day, or keep the date on the slide.

---

1 — Title (0:00 → 0:47)

On slide: "WireGuard for Apache NuttX." ESP32-S3 & SPRESENSE tunnel over real Wi-Fi · real WireGuard peers (Linux kernel, Windows client) · runs in a kernel build on QEMU (BUILD_KERNEL) · one source across NuttX 13.0.1 / master / 12.7.0.

Say:

- Today I will talk about adding WireGuard to Apache NuttX.
- It is a WireGuard VPN that works as a network device called wg0. The tunnel runs over real Wi-Fi on ESP32-S3 and on SPRESENSE, and the other side is always a real WireGuard program — the Linux kernel module and the official Windows client. The apps version builds from one source on three NuttX versions. Later in the talk, you will see the new driver running inside KERNEL and PROTECTED builds on QEMU, and in normal FLAT builds on both boards. That is the harder and more interesting half.
- This talk is about what “it works” was hiding, and what moving it into the kernel brought out.

2 — Agenda (0:47 → 1:21)

On slide: The seven parts, each a short title with a keyword line and its time: overview & background (~6 min) / the problem and the idea (~4) / the plan (~2) / version 1: the app, four hidden bugs (~5) / version 2: the kernel driver (~8) / live demo (~3) / summary & next steps (~4).

Say:

- Here is the plan for this talk.
- There are seven parts. First, an overview and the background: who I am, what NuttX is, and how this started. Second, the problem I wanted to solve, and the idea. Third, the plan. Fourth, version one, an app: the first tunnel, four hidden bugs, and the first demo. Fifth, version two, the kernel driver, and how I tested it. Sixth, a live demo with the StackChan. And last, a summary and the next steps. About thirty minutes, then questions.

3 — Agenda: ① Overview and Background (1:21 → 1:31)

Say (one line): Let's start with part one: an overview and the background.

Pause: point at the highlighted line, one beat.

4 — About Me (1:31 → 2:21)

On slide: Three bands: photo, name line and the fox icon / five hobby photos (space, semiconductors, DNA nanotech, robots, Linux) with three activity chips / a one-row path Robotics → Computer Vision → Edge AI → Embedded Systems → NuttX / WireGuard, a picture per step.

Say:

- First, a little about me.
- I am an edge AI engineer at Sony Semiconductor Solutions. I work with Apache NuttX from the application side. I am from Japan: born in Nagoya, university in Sendai, and now I live in Kawasaki. Outside work, I am a hobby engineer. I build things around space, semiconductors and robots, I made DNA nanostructures at university, and I love Linux. On social media, I am the fox in this picture. I often join hackathons, I help run software conferences, and I wrote a book about open source.

- My road to this project went through molecular robots, computer vision and AI systems, edge AI cameras, and embedded systems on SPRESENSE. NuttX and WireGuard are the latest stop on that road.

5 — Project Overview (2:21 → 3:01)

On slide: The wg0 netdev picture: application / NSH, the NuttX network stack, wg0 next to eth0 / wlan0, a UDP socket underneath, the Internet, the peer.

Say:

- Here is the whole idea in one picture.
- Apache NuttX is an RTOS that follows POSIX and has its own TCP/IP stack. It had no VPN. This project adds wg0 as a virtual network interface. Applications see a normal interface. Under it, the encrypted data travels over a UDP socket. The other side is a real WireGuard peer — Linux, Windows, or another gateway.
- Because NuttX follows POSIX and uses BSD sockets, it is a good fit for this work. FreeRTOS and Zephyr can run a VPN too. But on NuttX, a VPN can become a normal network device very naturally.

6 — How It Started: February to October (3:01 → 3:53)

On slide: A month axis Feb → Oct. Above it: mid Feb, read the GSoC idea list / early Mar, proposed the plan / Mar 20, heard about this conference from Alan, sent a talk / early Apr, GSoC proposal / early May, talk accepted, GSoC not selected / Oct, Glasgow. Below it: Jun–Jul a little at a time, August holiday most of the work, September FLAT app → kernel driver.

Say:

- Here is how this project went, month by month.
- In the middle of February, I read the GSoC idea list and started to look into it. In early March, I wrote my plan as an issue and proposed it. On March 20, Alan told me about this conference, so I sent a talk proposal. In early April, I sent my GSoC proposal.
- In early May, I got two answers. The talk was accepted, and the GSoC proposal was not. I decided to build it anyway.
- From June, I worked on it a little at a time. I did most of the work during the summer holiday in August. In September, I moved it from a FLAT app into a kernel driver. That is the second half of this talk.

7 — Background: What Is NuttX (3:53 → 4:41)

On slide: Software stack (applications / POSIX, ANSI, BSD socket API / networking, VFS, file systems, device drivers, NSH / NuttX kernel / MCU, MPU) and tags: RTOS, POSIX-oriented, BSD sockets, VFS, NSH, native networking, device drivers, ASF top-level project.

Say:

- First, what is NuttX?
- NuttX is a real-time operating system that puts standards first. It follows the POSIX and ANSI APIs, it uses BSD sockets for networking, and it has a real file system, a shell called NSH, built-in networking and device drivers. All of this is made small enough to run on microcontrollers, from 8-bit to 64-bit. It is also a top-level project of the Apache Software Foundation, and its community develops it in the open.
- The one point I want you to remember: NuttX is a small RTOS, but you program it like Unix. That is rare. And that is why almost everything later in this talk — sockets, network devices, a POSIX-style port — is possible at all.

8 — Where NuttX Fits (4:41 → 5:47)

On slide: FreeRTOS / Zephyr / NuttX table (design centre, typical use), GitHub stars with the note that stars are visibility, not deployment; number cards: 15+ architectures, 300+ boards, 1500+ configs, Sony, PX4, Xiaomi OpenVela 1000+ SKUs, Japan's 2024 lunar mission.

Say:

- Thirty seconds on where NuttX fits, and whether it is a toy.
- These are three good RTOSes, and each has a different focus. FreeRTOS is a small real-time kernel, on purpose, for MCU firmware and control. Zephyr is an all-in-one platform for connected devices, wearables and industrial IoT. NuttX is the Unix-like one: networking, file systems and drivers, used in drones and sensing. This is not a ranking. By GitHub stars, NuttX is the smallest of the three, and I want to say that clearly. But stars show how many developers notice a project, not how many devices use it.
- And it is really used. Upstream, NuttX supports more than fifteen CPU architectures, more than three hundred boards, and more than fifteen hundred configurations. In real products it runs in Sony devices, PX4 flight controllers, and more than a thousand Xiaomi OpenVela products, and it flew on Japan's 2024 Moon mission. It is small next to the big ones, but it is not a toy.

Check before the talk: the star counts move. Re-check them on the day, or leave the date on the slide as it is.

9 — History: NuttX at Sony (5:47 → 6:49)

On slide: Timeline: 2015 Sony audio products / 2018–19 SPRESENSE, CXD56xx (OSS + ELC Europe 2019) / 2020 upstream collaboration (NuttX Online Workshop, "A Journey from Fork to Mainline") / 2020s sensing and edge devices.

Say:

- This is Sony's history with NuttX. It is not my story yet, and it is older than edge AI.
- Sony has sold audio products running NuttX since around 2015. A few years later, SPRESENSE and its CXD56xx chip added audio, GNSS, camera and multi-core support to NuttX. In 2019, Masayuki Ishikawa from Sony presented this work at Open Source Summit and Embedded Linux Conference Europe. After that, Sony engineers became major upstream contributors. One of them, Alin Jerpelea,

later chaired the NuttX project management committee. His 2020 talk was called “A Journey from Fork to Mainline.”

- I am not saying that one engineer made Sony choose NuttX. I have no source for that, so I will not say it. What public talks and documents do show is simpler: Sony used NuttX for many years before edge AI, and over time it spread into camera, GNSS and sensing work. My own work builds on that.

10 — Experience: SPRESENSE and AITRIOS (6:49 → 7:46)

On slide: Photos of SPRESENSE and the AITRIOS edge AI camera; the flow "application development → system integration → deployment & operation".

Say:

- That was Sony's history. This is mine. Same hardware family, but I see it from a different place.
- On SPRESENSE, I use the NuttX SDK for sensing, camera, GNSS and LTE — prototypes and embedded development, on the application side. On AITRIOS edge AI cameras, which are built with ESP32 and NuttX, my job is to set up cameras and keep them running on site, not on a desk next to a debugger.
- In both cases, I come to NuttX from the same direction: first application development, then putting the system together, then setting it up and running it. I never had to write a NuttX driver for any of that — until this project. And the question that brought me here was not “How do I make NuttX run?” It was “How do I reach a NuttX device after it is installed?”

11 — Agenda: ② The Problem and the Idea (7:46 → 7:56)

Say (one line): That is the background. Part two: the problem I wanted to solve, and the idea.

Pause: point at the highlighted line, one beat.

12 — Problem: Reaching a Device From Far Away (7:56 → 8:48)

On slide: Engineer → Internet → router → Wi-Fi → NuttX device, showing that WPA2/WPA3 covers only the Wi-Fi hop; cards: open it to the Internet / make your own protocol / use a vendor's cloud.

Say:

- Now, the problem itself.
- Wi-Fi security is real, but it protects only one part of the path: the radio link between the device and the access point. Everything before that — the engineer, the Internet, the router — is outside what WPA2 or WPA3 can protect. What I needed was a secure path for the whole way, from the engineer to the device, not just the last few meters.
- Without that, every choice has a problem. You can give the device a public IP and hope. You can make your own secure protocol and keep it working forever. Or you can hand the connection to a vendor's cloud. The same problem appears in edge AI, industrial IoT, and remote systems in general. It is not limited to this project.

13 — Options: Other Ways to Reach a Device (8:48 → 10:05)

On slide: A table of seven ways (open a port / HTTPS to a cloud API / MQTT broker / reverse tunnel or relay / VPN on the router / carrier closed network / VPN on the device) against: who connects, any service or not, server in the middle, where security ends, what NuttX already has.

Say:

- Before I chose WireGuard, I compared the usual ways to reach a device.
- Opening a port is easy, but then the whole Internet can reach the device. HTTPS and MQTT are what most IoT products use. The device connects out to a server, so NAT is not a problem, and NuttX already has clients for both. But you only get the messages you designed — you cannot log in or open a web page on the device. And the security ends at the broker or the cloud.
- A reverse tunnel gives you one port through a relay server. A VPN on the router is great when the router is yours — but the last part of the path, on the LAN, is not protected, and at a customer's site the router is usually not yours. A carrier closed network works only over cellular, and you need a contract.
- A VPN on the device is the only row that has all three: any IP service, no server in the middle, and security all the way to the device. MQTT and HTTPS are still the right tools for data. This is for maintenance access.

If asked about Tailscale or ZeroTier: they are also a VPN on the device, but they usually need a bigger OS than a microcontroller has today (footnote on the slide).

14 — Solution: Why WireGuard (10:05 → 10:58)

On slide: Engineer / Linux / Windows ↔ WireGuard tunnel ↔ NuttX device; modern IP-layer VPN / UDP transport, public-key peers / already everywhere.

Say:

- So, why WireGuard?
- WireGuard is a modern VPN that works at the IP layer. It is small and simple by design, with much less code than IPsec or OpenVPN, and that matters a lot on a microcontroller. It runs over plain UDP, and each peer is known by its public key, not by certificates or a long setup. If a packet does not carry the right key, WireGuard does not answer at all — to everyone else, the device is just a brick wall. And most importantly, it is a standard. It has been in the Linux kernel for years, there is an official Windows client, and the other end can be any WireGuard peer.
- So I chose a standard protocol, instead of making my own or depending on a vendor's cloud.

15 — Architecture: WireGuard on the NuttX Network Stack (10:58 → 11:46)

On slide: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0 / usrsock → network, with TX and RX arrows.

Say:

- This is one step more technical than the overview. This is where the implementation starts.
- Going out, a packet comes down from the application through TCP or UDP and IP, and reaches wg0 like any other network device. WireGuard encrypts it, and it leaves as a UDP packet — through wlan0 on a board, or through usrsock when the Wi-Fi chip itself works through sockets. Coming in is the same path the other way: a UDP packet arrives, WireGuard decrypts it, and it is passed up to wg0 as if it came in on a normal interface.
- From here on, the work is building this middle part — wg0 and the encryption — without changing the layers above or below it.

16 — Agenda: ③ The Plan (11:46 → 11:56)

Say (one line): Part three: the plan. How do you reuse an existing WireGuard?

Pause: point at the highlighted line, one beat.

17 — Porting Strategy: Reusing wireguard-lwip Without lwIP (11:56 → 12:57)

On slide: Three cards: protocol & crypto (KEEP) / platform hooks (ADAPT, the four functions in wireguard-platform.h) / lwIP network glue (REPLACE).

Say:

- Why I started from smartalock/wireguard-lwip, and did not write the crypto myself.
- wireguard-lwip is cleanly split into three layers. The protocol and crypto — the handshake, the data transport, BLAKE2s, ChaCha20-Poly1305 and Curve25519 — are portable C that does not depend on any OS. I used the crypto without changes. The protocol core needed two small fixes later, after a review, and they are written down in PATCHES.md. Everything that depends on the OS was already kept behind one header, wireguard-platform.h, as four functions. I only had to replace the lwIP network glue — the part that registers an lwIP netif and handles lwIP buffers — because NuttX does not have lwIP at all.
- I did not have to rewrite the protocol or the crypto, and that was not luck. That is what a clean line between the protocol and the OS gives you. Because of it, this port took weeks, not months.

18 — Design: Mapping lwIP onto NuttX netdev (12:57 → 13:19)

On slide: netif → net_driver_s, pbuf → iob, netif_add() → netdev_register(), callbacks → devif_poll().

Say:

- This was the most important porting decision.
- Instead of bringing lwIP into NuttX, I matched each lwIP idea to NuttX's own. netif becomes net_driver_s, pbuf becomes iob, netif_add becomes netdev_register, and the send callback becomes NuttX's devif_poll. wg0 is registered as a TUN-type network device, and the “wire” under it is a UDP socket.

19 — Agenda: ④ Version 1: The App (13:19 → 13:29)

Say (one line): Part four: version one, an app. It worked, and then the bugs came.

Pause: point at the highlighted line, one beat.

20 — Result: A Working Tunnel Is Only the Beginning (13:29 → 14:14)

On slide: Protocol & crypto: crypto byte-identical, protocol core with two small fixes (PATCHES.md) / OS hooks: four functions / network glue: the real work. wg genkey / set / setconf at run time.

Say:

- Here is the result of the first version.
- More than three thousand lines of protocol and crypto came over. The crypto came over exactly as it was. The protocol core needed two small fixes after a review — and one of them was a bug in wireguard-lwip itself: its rate limit never worked. The four OS functions are small. The real work was the network glue. And it worked: you can make a key, set peers and load a config, all at run time, with no rebuild. So far, a success story — but the princess was in another castle. The rest of this talk is about what that success was hiding.

21 — The Testing Pattern: Simple Tests Pass, Deep Tests Fail (14:14 → 14:42)

On slide: SIMPLE TEST (handshake / ping, PASS) → ASSUMPTION ("that part must be fine") → DEEPER TEST (TCP / real hardware / kernel / security, FAIL).

Say:

- This is the main point of the talk.
- Every hard bug in this project looked the same. A simple test passes, but a deeper one fails. Even worse, the passing test makes you believe that part is fine, so you look everywhere else for hours. The next slides are examples of this pattern — four in the apps version, and two more after I moved it into the kernel.

22 — Bug 1/6: The Detached pthread (First Symptom: recvfrom Never Returned) (14:42 → 15:48)

Say:

- The first one: something I believed about a thread's life that was not true.
- The first symptom was simple: recvfrom never returned, the timer never ran, and no handshake was sent. I blamed the receive timeout option and switched to poll, but still nothing was sent. The real cause was here. The receive thread was created as detached. When the command that started it finished, the thread stopped too — it did not even appear in ps. When I added a short sleep after creating it, its printf ran, so it really was created. My mistake was thinking that “detached” means “it

lives longer than the command." Detach only decides when the thread's memory is freed, not whether it keeps running after the command exits. So when the command ended, the thread ended with it. That is why, in the kernel version, the driver itself owns the thread. And NuttX's UDP code does use that timeout option; I simply never tested the old path again.

23 — Bug 2/6: ping Works, TCP Dies (15:48 → 16:40) *(my favorite)*

Say:

- This is my favorite bug in the project.
- On the same tunnel, ping was perfect, and even a TCP handshake finished — and then the connection died. When I added logging to the send path, it showed EBADF: bad file descriptor. All the sends that worked happened inside the receive task itself — handshake replies, keepalives, ping replies, the SYN-ACK. Sends from any other task failed.
- The reason: in NuttX, a file descriptor belongs to one task group. It is not shared by the whole system. The fix was to stop using file descriptors and use NuttX's internal socket API. It works on a struct socket, which is just memory, so any task can use it. That is exactly why the kernel version keeps the socket as a struct socket.

24 — Bug 3/6: "No Real Impact" Was 10× (16:40 → 17:20)

Say:

- The third: something I never measured.
- The fix without file descriptors also broke poll, because poll works with file descriptors. So, as a quick fix, I checked the socket every fifty milliseconds. I thought the only cost was waking up the CPU when it was idle. I was wrong. Every packet that arrived waited up to fifty milliseconds, and that alone limited the speed — about ten times slower than it should have been. I thought it had no real cost, and it cost ten times. The real fix was a proper wait: a callback on psock_poll and a semaphore.

25 — Bug 4/6: sim Passes, Real Hardware Is Different (17:20 → 18:14)

Say:

- The fourth, and it will come back later: stack sizes nobody had measured.
- Two things only real hardware showed me. First, saving the config into a file through the shell froze the board — but only on the flash file system of real hardware, never in sim. I fixed it with a wg saveconf command that writes the file itself.
- Second, the receive task's stack. That task calls into the IP stack on its own stack, so the whole TCP call chain sits on it, and for packets it answers right away, so does encrypting the reply. The default size, 3072 bytes, was a number nobody had measured. When I measured it on the board with stack coloration and ps, it was ninety-four percent full — only 176 bytes left. Please remember this one.

26 — Version 1 Demo: The First Video (18:14 → 18:40)

On slide: Left: a still frame of the video (the Windows WireGuard client beside a telnet session into NuttX on the ESP32-S3). Right: the steps telnet 10.10.0.2 / uname -a, ifconfig / webserver & / browser http://10.10.0.2/; ESP32-S3 DevKit, apps version (FLAT), USB unplugged; the official Windows client; small text youtu.be/1kyX2av5WG4.

Say:

- Here is a frame from the version one video: I logged in with telnet through the tunnel, ran a few commands, and opened a small web page on the board. The board runs on a power adapter only, and the other end is the official Windows WireGuard client. So version one was enough for a demo; next, I made it a real NuttX driver.

No playback. The still is enough; the video is the backup for the live demo.

It is the apps version. Do not mix it with the kernel-version results later in the talk.

27 — Agenda: ⑤ Version 2: The Kernel Driver (18:40 → 18:50)

Say (one line): That was version one. Part five, the harder half: a real kernel driver, and how I checked it.

Pause: point at the highlighted line, one beat.

28 — Part 5: Into the Kernel (18:50 → 19:26)

Say:

- This is the work I did after the first version, and it is the part that NuttX needs before this can go upstream.
- Version one lives in apps and uses socket internals. That is fine for a FLAT build. But it can never be a proper in-tree driver, and it does not work in PROTECTED and KERNEL builds, where user space and the kernel are kept apart. So I moved the device into the kernel, under drivers/net/wireguard, and I control it from a small user-space wg command through ioctl.

29 — The Kernel Design (19:26 → 20:23)

Say:

- This is the shape of the real driver.
- wg0 is now a lower-half network device in the kernel. It owns its UDP socket and its receive thread, so the socket and thread problems from version one cannot happen, by design. The settings go through a simple ioctl ABI with no pointers. No pointers: the kernel reads the one struct the caller passed, during the call, and never has to follow a pointer into user memory. The private key is write-only: no ioctl ever gives it back — but it is still in the config file on the user side. The crypto in the kernel is NuttX's own. The protocol core is still derived from wireguard-lwip, under BSD-3-Clause, so

the driver needs ALLOW_BSD_COMPONENTS, and I say so in the LICENSE file. The user command also has a small MIT-licensed X25519, for making keys offline.

30 — Bugs 5 & 6/6: Same Shape, One Level Deeper (20:23 → 21:28) *(locked with the kernel design)*

Say:

- And the same pattern came back in the kernel — twice.
- First, a bug in NuttX's own ChaCha20-Poly1305: the counter was written into the wrong bytes of the nonce. When the counter is zero, both layouts are the same, so the handshake works, and then every data packet after the first one fails to decrypt. Simple test passes, deep test fails. Nothing in NuttX had called this code before, so nobody had hit it. It is a two-line fix, and it goes upstream as its own crypto PR. So porting to NuttX found a hidden bug in NuttX itself.
- Second, a crash that happened only in a kernel build. Setting the key copied the peers into a six-kilobyte array on the stack, but the kernel stack is only three kilobytes. Overflow, a broken heap, panic. In sim, with big task stacks, it never showed up. It was the same stack lesson as before, one level deeper. I moved the array to the heap.

31 — Testing: Seven Places It Must Work (21:28 → 23:37) *(core — do not cut)*

On slide: Table (where / version / what I checked): sim / rv-virt knetnsh64 (BUILD_KERNEL, QEMU) / rv-virt pnsh64 (BUILD_PROTECTED, QEMU) / rv-virt knetnsh64_smp (4 CPUs, QEMU) / SPRESENSE measured (FLAT) / ESP32-S3 + SPRESENSE over real Wi-Fi (FLAT; both board rows before the final rebase) / Version 1 (app), v0.1.1 on both boards.

Say:

- Let me be exact about which version was checked where.
- The kernel driver builds with no errors in the sim config — the driver PR adds that config, so CI will build the driver. It also runs a full tunnel in a kernel build in QEMU, rv-virt knetnsh64, to WireGuard in the Linux kernel. The wg command is a separate program, and it reaches the driver through system calls.
- Next, PROTECTED, also in QEMU. The ioctl ABI has no pointers inside, so the kernel only touches the one struct the caller passed. With the memory split on, every ioctl crosses the boundary, and the tunnel reaches Linux. But in repeated runs, two out of eight hit a fault in user space. I have not found the cause yet, so it is an open problem, not a pass.
- Then, locking: four CPUs in QEMU, a flood of packets, while another CPU changes the settings and does down and up. No fault, and the tunnel came back. But this is one scripted run, not a full search for race conditions.
- The SPRESENSE row is the one I would show a reviewer first. The simulator's memory numbers do not apply to a board, so I measured the board itself: two minutes of traffic, then forty down-and-up cycles over real Wi-Fi. Heap, buffers and every stack: exactly the same before and after. The receive

thread used one thousand four hundred and seventy-two bytes, out of a six-kilobyte default. That default used to be a guess. Now it is a measurement.
- The kernel driver also runs on both boards over real Wi-Fi, with the official Windows client: native Wi-Fi on the ESP32-S3, and usrsock on SPRESENSE. These are normal FLAT builds, from the version before the final rebase. KERNEL and PROTECTED ran only in QEMU, and the KERNEL and SMP tunnels were run again after the rebase. The last row is version one, the app, on both boards.

32 — Testing: A Working Tunnel Does Not Prove Enough (23:37 → 25:09) *(core — do not cut)*

On slide: Three rows (same private key every boot / session keys that outlive the tunnel / a leak that shows after hours), each with "would ping notice?" → no, and where it was measured (SPRESENSE / sim / sim).

Say:

- The last slide was about where it runs. This one is about what a working tunnel cannot tell you.
- First, keys. NuttX can give /dev/urandom an xorshift128 backend, with a seed from build-time settings. Then a board gives out the same private key every time it starts, and the handshake, the ping and the demo all still work. So I reset a SPRESENSE three times — resets, not power cycles — and got three different keys. That is a quick check, not a proof of good randomness.
- Second, in the simulator: I read the device's memory while the tunnel was up, and again after down. The session keys were all zero after down. The static key stays, on purpose, so that up works again. There is exactly one copy of it, and that is written in the driver's documentation.
- Third, the long test, also in the simulator: a hundred and fifty-one forced rekeys past the key lifetime, ten endpoint changes, and a hundred down-and-up cycles. The buffers all came back, and the live allocations went back to exactly the start-up number. The receive thread's stack peak was two and a half kilobytes in the simulator — one and a half on the board — against a six-kilobyte default. What I am not saying is that it ran for a week, or that the simulator's memory acts like the board.

33 — A Design Choice: The Timestamp Problem Has No Easy Answer (25:09 → 27:05) *(core — do not cut)*

On slide: What replay protection obliges, the four options (real-time clock + in-boot high-water mark / persist every time / durable range reservation / wait to catch up), and the measured pair: clock unset → no handshake in a 75 s watch, clock set → 4.1 s.

Say:

- This is the one place where I had to decide, not test. And then I measured my decision.
- WireGuard's replay protection gives the side that starts the handshake a rule: every handshake needs a newer timestamp than the peer has seen — even after a reboot. The device is usually the side that starts, and it is also the side least likely to have a clock with a battery.
- Doing nothing does not work. With an uptime-based timestamp, a device is cut off for as long as it ran last time: a device that ran for a month cannot reconnect for a month. Saving every timestamp wears out the flash. Reserving a range in flash is the right long-term answer, but it needs a write

guarantee I could not verify on these file systems. Shipping that would look solved without being solved. Here, the easy choice and the right choice were not the same.

- So I chose the real-time clock plus the highest value used in this boot — exactly what Linux and wireguard-go do. What I added: if the clock was never set, the driver prints a warning once, and says what will happen.
- And I measured it on the board. The obvious test passes: reset the board, ping it, and the tunnel is back — because your ping made the peer start the handshake. So I sent the board nothing at all. Same board, one change. With the clock as it boots: a warning, and no handshake in a seventy-five-second watch. With the clock set: a handshake in four point one seconds. Without a clock, the device may not reconnect by itself, and a ping test will not show it. That is why the driver prints a warning.

34 — Agenda: ⑥ Live Demo (27:05 → 27:15)

Say (one line): That was all engineering. Part six: what it actually looks like.

Pause: point at the highlighted line, one beat.

35 — Demo Setup: One Tunnel on a Phone's Network (27:15 → 28:13)

On slide: Diagram: venue Wi-Fi (blocks devices from talking to each other) → phone hotspot (its own small local network) → laptop (WireGuard peer in Docker, wg0 10.10.0.1; voice clips on :8000) and StackChan (ESP32-S3 + NuttX, wg0 10.10.0.2 over wlan0 on the hotspot; apps version v0.1.1, FLAT build), joined by one WireGuard tunnel; SPRESENSE as a dashed second peer (wg0 10.11.0.2).

Say:

- Here is the demo setup.
- My laptop and the StackChan are both on my phone's hotspot. The venue Wi-Fi works, but it blocks devices from talking to each other, so the phone makes a small local network for us. The phone can use the venue Wi-Fi or mobile data for its own Internet, but the demo does not need it.
- Between the laptop and the StackChan there is one WireGuard tunnel. The laptop side is a normal Linux WireGuard in a container, at 10.10.0.1. The StackChan is 10.10.0.2. Everything in the demo — telnet, the web page, and even the voice — goes through that tunnel. I made the voice clips on the laptop in advance, so there is no cloud API anywhere.
- Please watch two things: the addresses after I log in, and the packets. Outside the tunnel, you only see encrypted UDP.

36 — The Demo (28:13 → 29:43)

On slide: Seven steps: wg show / telnet 10.10.0.2 / ifconfig / stackchan face happy / stackchan say hello.wav / tcpdump outside vs. inside / wg show again. Right panel: outside = only encrypted UDP 51820, inside = the telnet text; "Runs the apps version (v0.1.1), FLAT build."

Say:

- The point of this demo is that there is nothing special to see. The StackChan is just a normal computer at the far end of an encrypted tunnel. This board runs the apps version — the same app as in the video. The kernel driver ran on this chip too, on slide 31, but the robot demo is built on the app.
- First, wg show on the laptop: the handshake is done, and you can see the StackChan's address on the phone's network. Then I telnet to 10.10.0.2 — that is the address inside the tunnel. When I type ifconfig, you see both: wlan0 on the phone's network, and wg0 inside the tunnel. So I am really logged in through the tunnel.
- Now I can control it. Let's make it happy, and let it say hello. The voice comes from the laptop through the same tunnel.
- Last, the packets. On the left, the phone's network: only encrypted UDP. On the right, inside the tunnel: the same telnet session, in plain text. And wg show again: the byte counts went up. Mischief managed.

Live: `scripts/stackchan/demo.sh`: `login` in the left terminal, `outside` and `inside` in two terminals on the right, `show` before and after. Put the terminals on the left and the StackChan (camera or on the desk) on the right.

Before the talk: phone hotspot on (2.4 GHz / maximize compatibility), laptop on it, Docker Desktop running, the StackChan joined with `nsh_wifi.py` (the passphrase is not saved on the board), `wg_setup.py` run once. Check `stackchan say` once. If the hotspot was turned off and on, its addresses may change: run `nsh_wifi.py` and `wg_setup.py` again. Rarely (about 1 in 20) the board stops during `say`; power it off and on, then repeat the same steps.

Never on screen: `wg showconf` (prints the private key), the key files in `%USERPROFILE%\stackchan-wg`, the hotspot passphrase.

Fallback: the recording. Switch at once if the live demo stalls.

37 — Agenda: ⑦ Summary and Next Steps (29:43 → 29:53)

Say (one line): Last part: what comes next, and what to take home.

Pause: point at the highlighted line, one beat.

38 — Running It for Real (29:53 → 30:27) *(cut candidate 3)*

Say:

- Going from “it works in a demo” to “you can really run it.”
- Keys are no longer built into the firmware. You make them and set them at run time, and they never end up in the firmware image. And for long runs, I check four separate signals every minute — is the router side alive, is the board on the network, is WireGuard carrying traffic, is the application data moving — so when it goes quiet, I can say which part failed instead of guessing.

39 — Other CPUs (30:27 → 30:55) *(cut candidate 2)*

Say:

- Did the decision from the start work?
- Keeping the OS behind a few functions was a bet. Here is the check, with the apps FLAT version: four CPU types — x86, ARM Cortex-A7, Xtensa on the ESP32, and Cortex-M4F on SPRESENSE — and no code change to move between them. The kernel driver does not use these four functions, so this check is for the app. The bet paid off.

40 — Giving Back (30:55 → 31:50) *(cut first)*

Say:

- Let me end this part with the values of this community.
- I will send the work in steps. First, a small crypto PR for the nonce fix, because it is useful on its own and helps everyone. Then the driver PR for the kernel device, and then the command PR for the user-space tool. The work is already split into small PRs, and every commit passes the patch checks and builds on its own. The design discussion and the upstream PRs are the next step; they have not started yet. Before I send them, every commit still needs my sign-off and an Assisted-by line, because AI tools helped me, and the final series still needs a run on real hardware. Community over code means being honest about where the work really is, and then letting review change it.

If asked "is it merged?": not yet. These are submission candidates, not a claim that it is mergeable. See the Q&A file.

If time is short: skip the whole slide. The talk's argument (the pattern, the verification) does not rest on it; just keep the answer above ready for questions.

41 — Summary: What I Built, and Three Things to Remember (31:50 → 32:48)

Say:

- To sum up.
- I built wg0, a WireGuard network device inside the NuttX kernel. It works on an ESP32-S3 and on SPRESENSE with the official Windows client, it talks to Linux kernel WireGuard in the simulator and in QEMU, and it is split into small PRs, ready for review. Nothing is sent upstream yet.
- And three things to remember. First, simple tests pass and deep tests fail — so design your tests to go deep: long data runs, real hardware, a real kernel build, not just a handshake. Second, a clean line between your code and the OS is worth the effort — in the apps version, four CPU types and three NuttX versions, with no code changes. Third, porting to a project is a chance to make it better. One bug was in NuttX's own crypto, and that fix will go upstream by itself.

42 — Thanks: The People Behind This Talk (32:48 → 33:23)

On slide: Four boxes: Alan / the Apache NuttX community / the Apache Software Foundation and the CoC organizers and volunteers / Sony's NuttX developers, past and present.

Say:

- Before I finish, some thanks.
- First, Alan, who told me about this conference and encouraged me to submit a talk. Second, the Apache NuttX community: when I wrote my GSoC proposal, people talked with me and encouraged me, and that is why I kept going after it was not selected. Third, the Apache Software Foundation and everyone who organizes Community Over Code. And last, the Sony engineers before me, who brought NuttX into our products and gave so much back to the project. This work stands on theirs.

43 — Thank You (33:23 → 33:32)

Say:

- Thank you. I am happy to take your questions — about the kernel driver, the bugs, the ioctl ABI, or the hardware.

After: keep coc-glasgow-qa.md at hand, not a slide. The usual questions (rebase status, #14, IOB exhaustion, checkpatch) are answered there and on no slide.